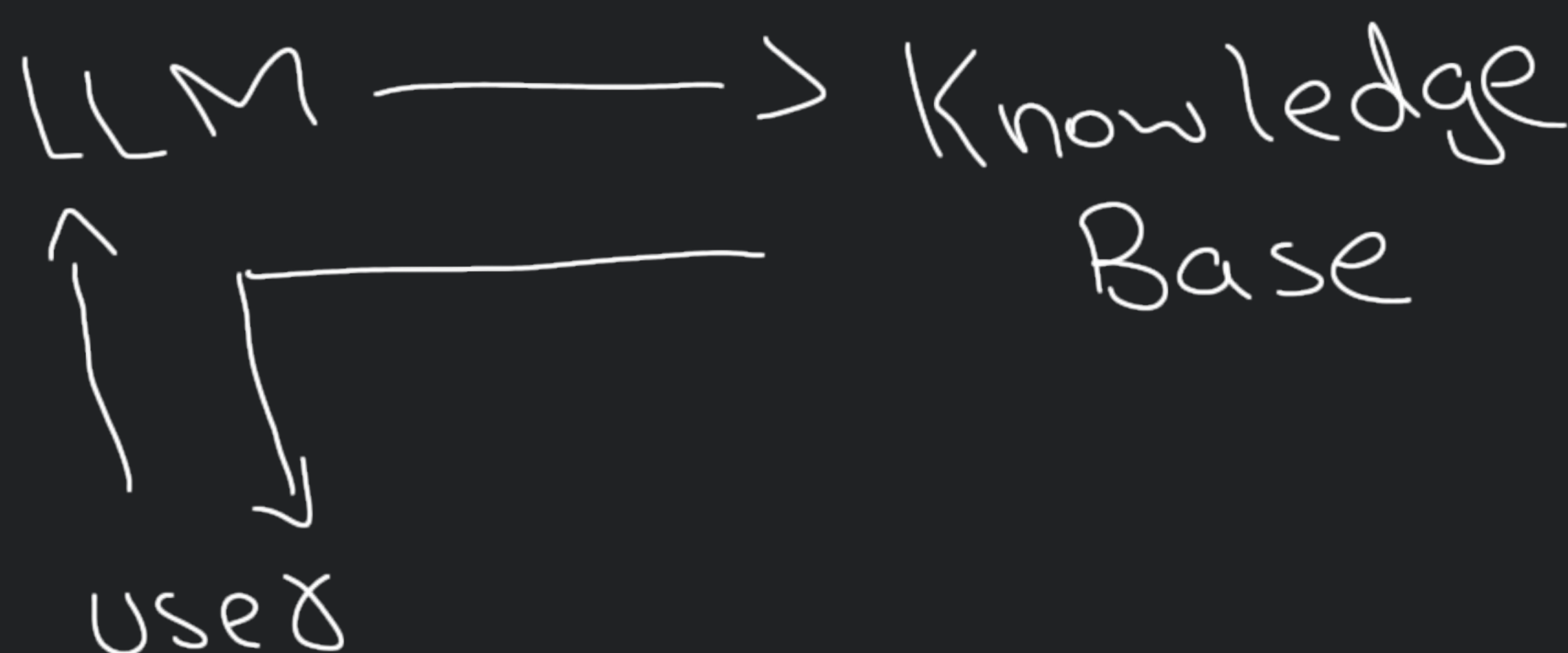


# RAG

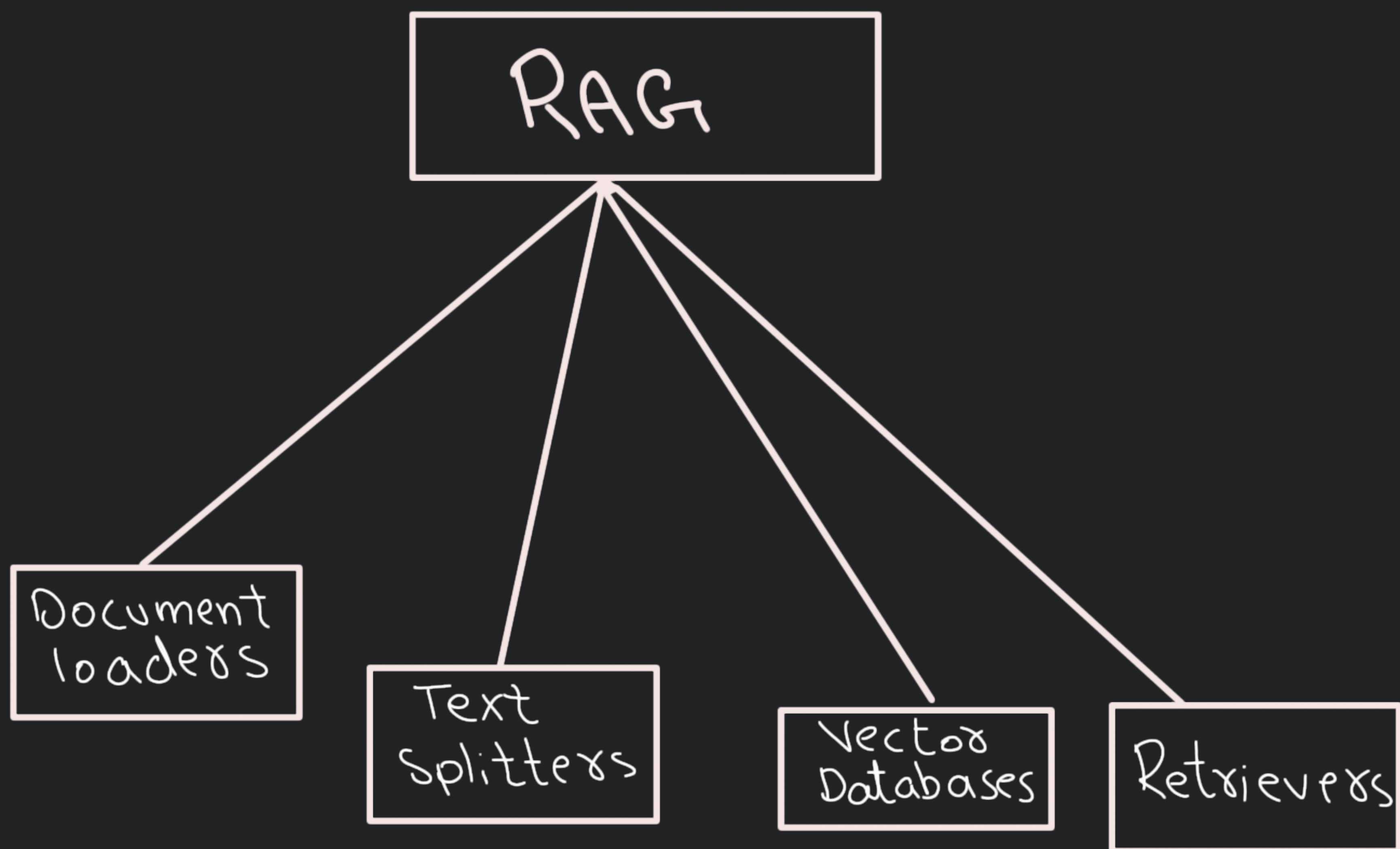
technique that combines information retrieval (from external knowledge base) with language generation (using LLM), where a model retrieves relevant documents from a knowledge base and uses them as a context to generate accurate and grounded responses.

## Benefits

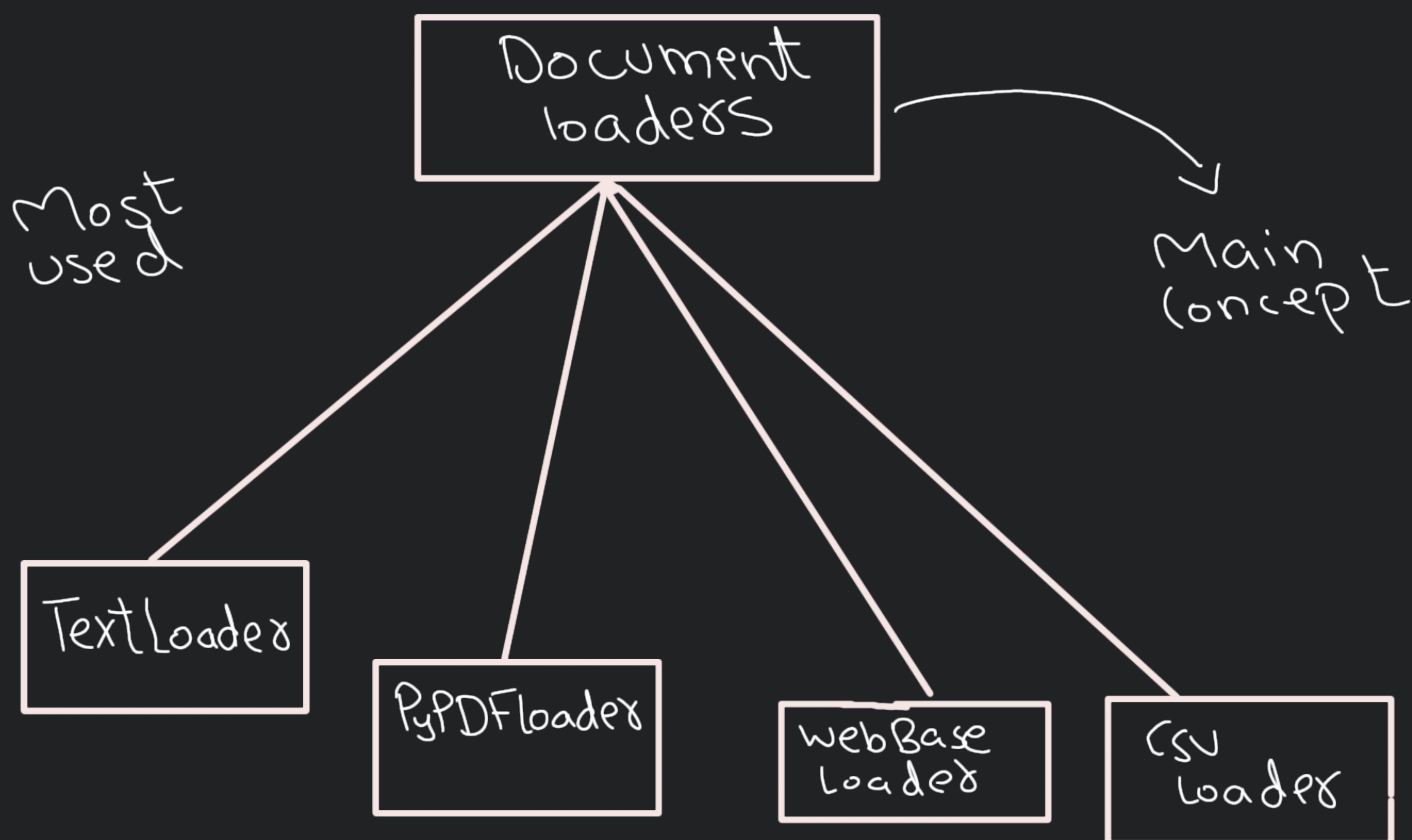
1. Use of up-to-date information
2. Better privacy
3. No limit of document size







## Document Loaders (there exists 100 of them)





Components in LangChain used to load data from various sources into a standardized format (usually as Document objects), which can be used for chunking, embedding, retrieval and generation

→ Document (  
page\_content = "The actual text content",  
metadata = {"source": "filename.pdf", ... }  
)

## 1) Text Loaders

Reads plain text (.txt) files and converts them into LangChain Document objects.

**Use Case:** ideal for loading chat logs, scraped text, transcripts, code snippets, or any plain text data



into a langchain pipeline.

## Limitation:

works only with .txt files



## LangChain:

```
from langchain_community.document_loaders  
import TextLoader
```

```
loader = TextLoader('cricket.txt', encoding =  
                    'utf-8')
```

```
docs = loader.load() // loads text file  
                     as a list of  
                     document
```

// every document is kind of  
divided into multiple paths and  
stored in a list

```
print(docs[0])
```

langchain  
document  
object



docs[0].page\_content // extract  
Page content

docs[0].metadata // extract  
metadata

## 2) PyPDF Loader (most used)

load content from PDF files and  
converts each page into a  
Document object

→ works on page-by-page basis

pages  
[25] → pypdf → 25 document  
objects

[  
Document(page\_content="Text from page 1",  
metadata={"page": 0, "source": "file.pdf"}),  
.....  
]



## Limitation:

→ uses PyPDF library under the hood -  
not great with scanned PDFs or  
complex layouts

| Use Case                       | Recommended Loader                               |
|--------------------------------|--------------------------------------------------|
| Simple, clean PDFs             | PyPDFLoader                                      |
| PDFs with tables/columns       | PDFPlumberLoader                                 |
| Scanned/image PDFs             | UnstructuredPDFLoader or AmazonTextractPDFLoader |
| Need layout and image data     | PyMuPDFLoader                                    |
| Want best structure extraction | UnstructuredPDFLoader                            |

## LangChain:

```
from langchain_community.document_loaders  
import PyPDFLoader
```

```
loader = PyPDFLoader(".....pdf")
```

```
docs = loader.load()
```

```
print(len(docs)) // Number of pages in  
pdf
```



### 3) Directory Loader

Loads multiple documents in a directory

LangChain:

```
from langchain_community.document_loaders  
import PyPDFLoader, DirectoryLoader
```

```
loader = DirectoryLoader(  
    path = 'books',  
    glob = '*.pdf',  
    loader_cls = PyPDFLoader  
)
```

```
docs = loader.load()
```

```
print(len(docs)) // Number of pages in  
all pdfs combined
```



#### ✓ load()

- Eager Loading (loads everything at once).
- Returns: A list of `Document` objects.
- Loads all documents immediately into memory.
- Best when:
  - The number of documents is small.
  - You want everything loaded upfront.

#### 🌀 lazy\_load()

- Lazy Loading (loads on demand).
- Returns: A generator of `Document` objects.
- Documents are **not all loaded at once**; they're fetched one at a time as needed.
- Best when:
  - You're dealing with large documents or lots of files.
  - You want to **stream** processing (e.g., chunking, embedding) without using lots of memory.

## 4) WebBase Loader

load and extract text content from web pages (URLs)

→ uses BeautifulSoup under the hood to parse HTML and extract visible text

### Use when:

For blogs, news articles, or public websites where the content is primarily text-based and static

### Limitations:

→ Doesn't handle JavaScript-heavy pages well (use SeleniumURLLoader)



for that)

→ Loads only static content (what's in the HTML, not what loads after page renders)

## LangChain:

```
from langchain_community.document_loaders  
import WebBaseLoader
```

```
url = "....."
```

```
loader = WebBaseLoader(url)
```

```
docs = loader.load()
```

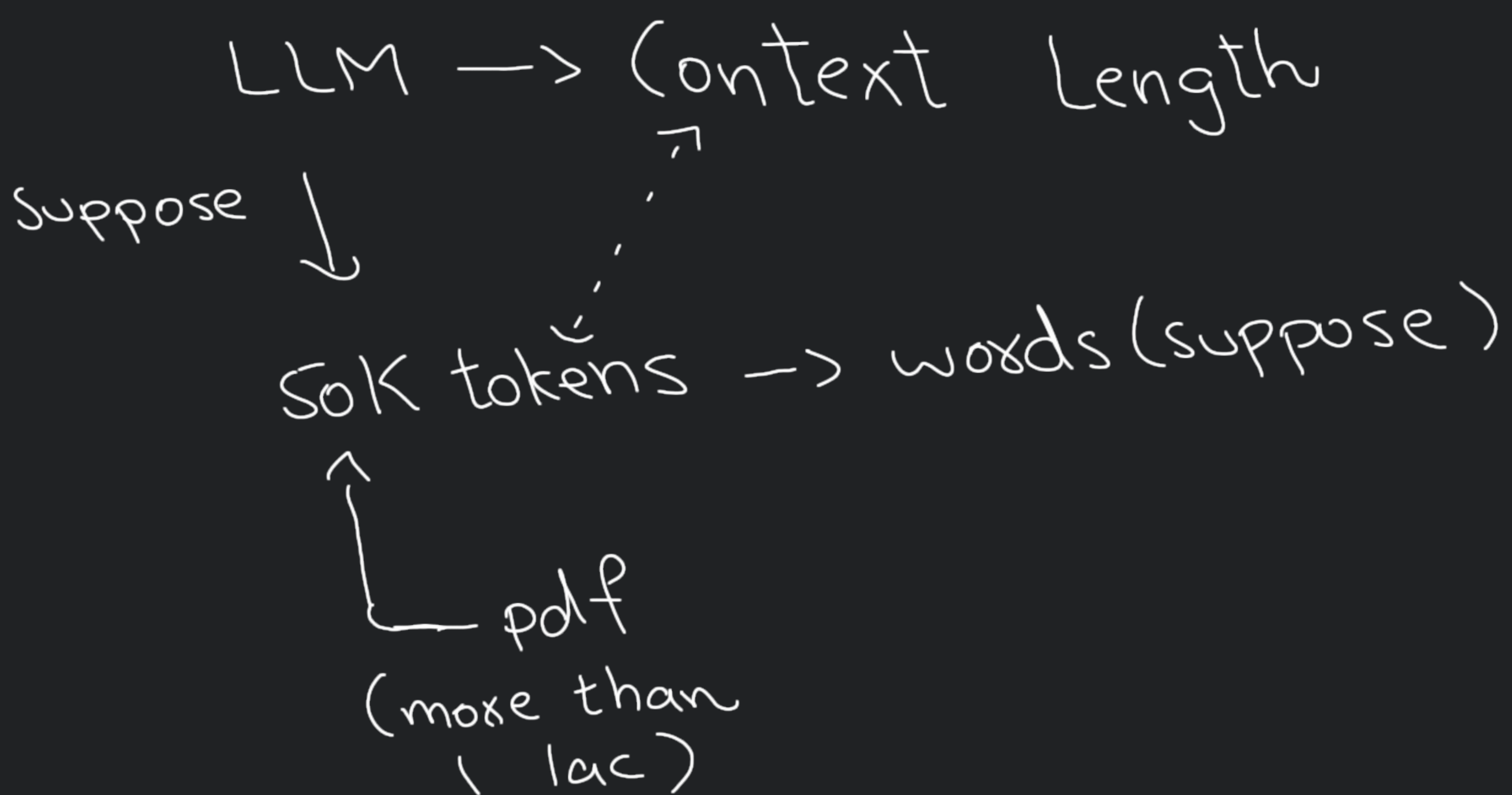
## 5) CSV Loader

load csv files into LangChain  
document objects — one per row  
(by default)



# Text Splitters

Process of breaking large chunks of text (like articles, PDFs, HTML pages, or books) into smaller, manageable pieces (chunks) that an LLM can handle effectively



## • Overcoming Model Limitations

Many embedding models and language models have maximum input size constraints. Splitting



allows us to process documents that would otherwise exceed these limits

## • Downstream Tasks

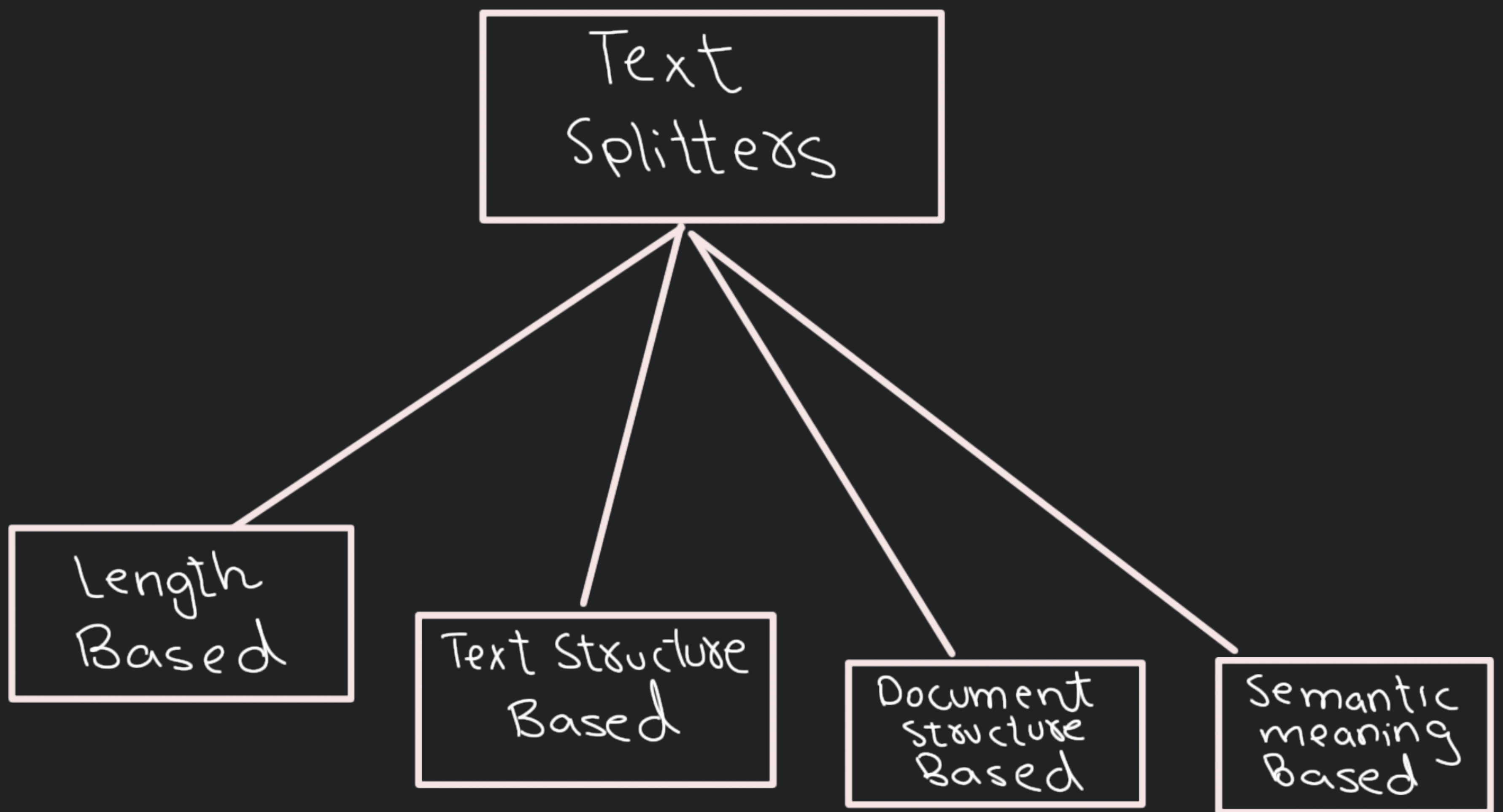
Text splitting improves nearly every LLM powered task

| Task            | Why split helps                                 |
|-----------------|-------------------------------------------------|
| Embedding       | Short chunks yield more accurate vectors        |
| Semantic Search | Search results point to focused info, not noise |
| Summarization   | Prevents hallucination and topic drift          |

## • Optimizing computational resources

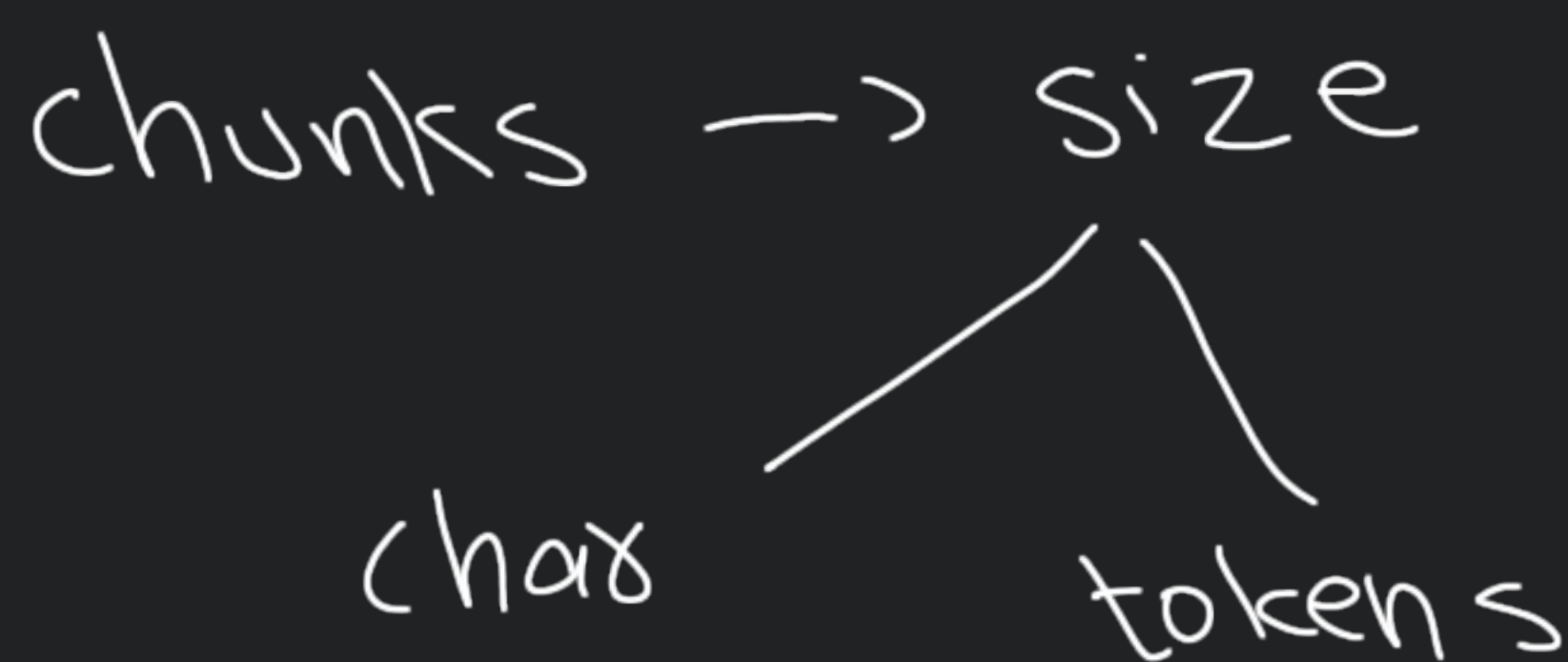
Working with smaller chunks of text can be more memory-efficient and allow for better parallelization of processing tasks





## 1) Length Based

Predetermine size of each chunk

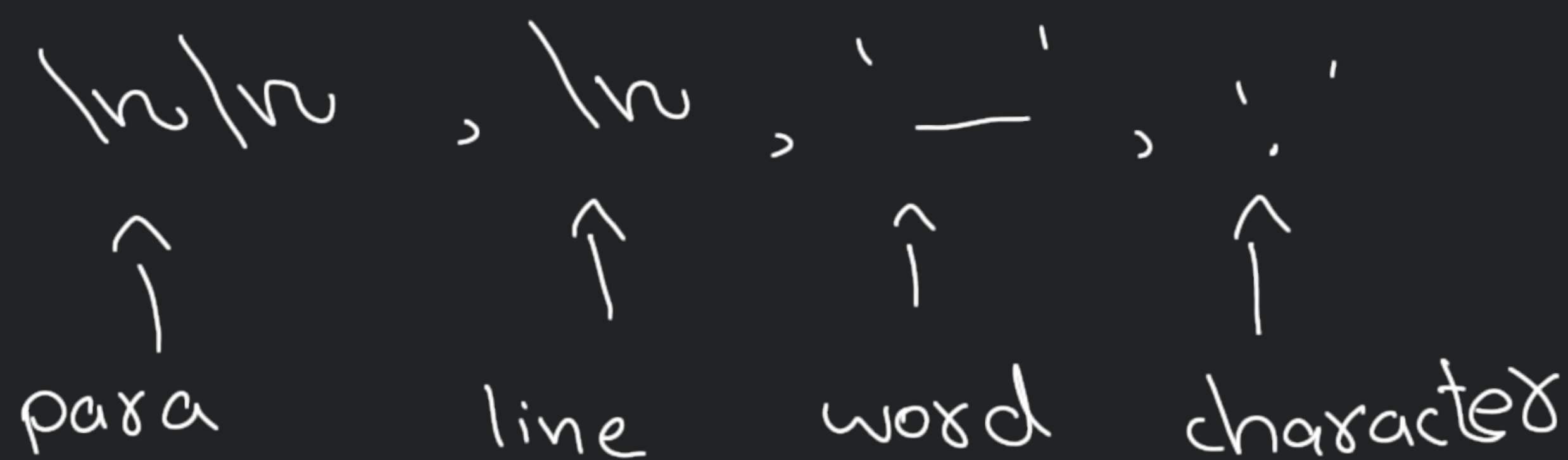


→ Might cut a word or paragraph  
than embedding wouldn't capture  
complete semantic meaning



## 2) Text-Structure Based

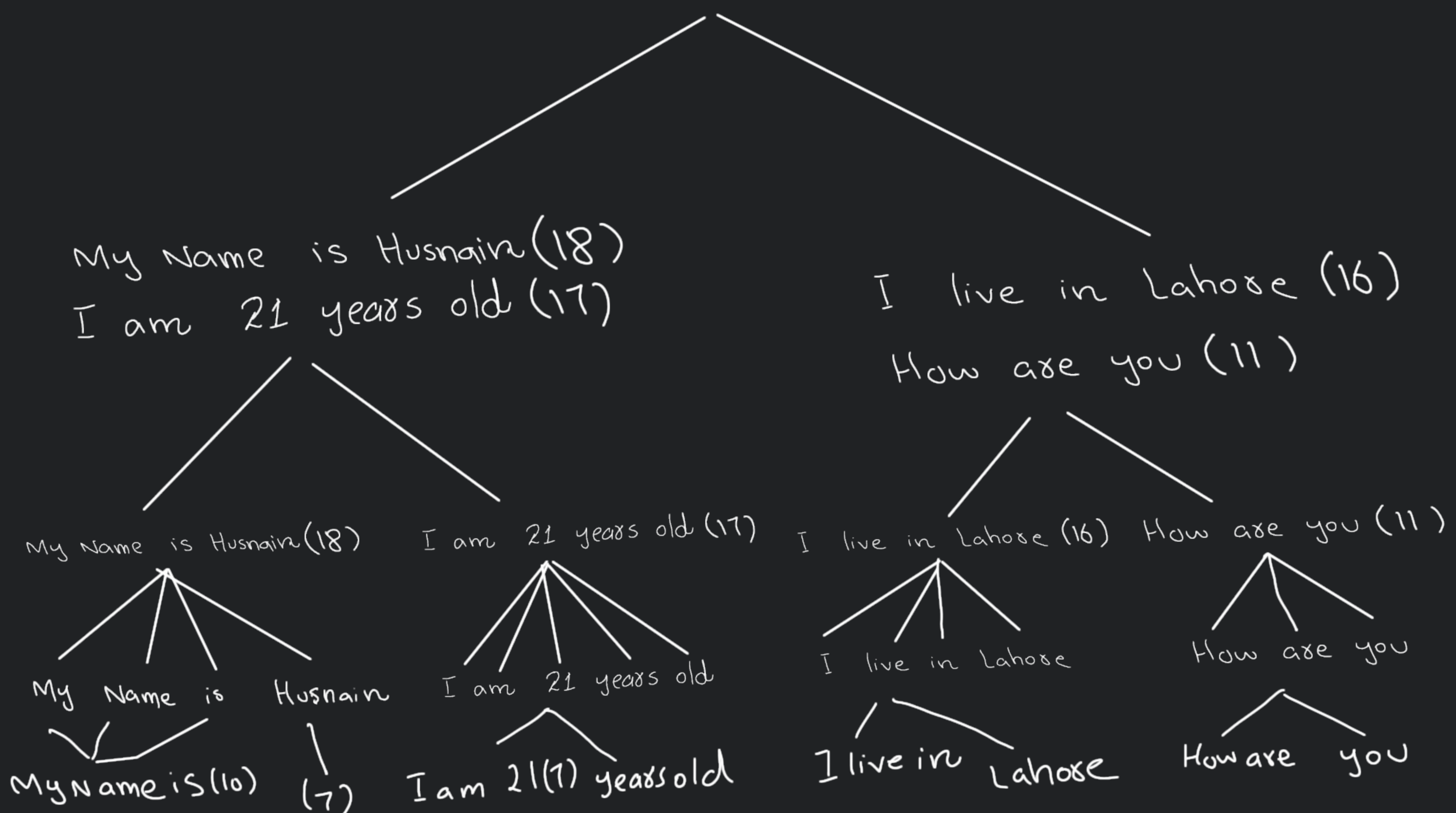
Recursive character text splitting technique



Example:

[chunk\_size=10]

My Name is Husnain (18)  
I am 21 years old (17)  
I live in Lahore (16)  
How are you (11)





Chunks:

[My Name is], [Husnain], [I am 21], [years old], [I live in],  
[Lahore], [How are], [you]

### 3) Document-Structured Based

Recursive character text splitting  
but with different splitters

### 4) Semantic Meaning Based

Generate embedding vectors for  
each sentence and compare  
their vectors to find similarities



# Vector Stores

Embedding: extract semantic meaning of a text and represent in numbers or vectors

→ Plot vectors of each text in  $n$ -dimensional space to find closest text

Challenges:

- 1) Generate Embedding vectors
- 2) Storage (can't be stored in relational DB)
- 3) Intelligent Semantic Search



# Solution:

## Vector Store

System designed to store and retrieve data represented as numerical vectors

## Features:

- 1) Storage - ensures that vectors and their metadata are retained, whether in memory for quick lookups or on-disk for durability and large-scale use
- 2) Similarity Search - helps retrieve the vectors most similar to a query vectors



3) Indexing - provide a data structure or method that enables fast similarity searches on high-dimensional vectors (e.g, approximate nearest neighbor lookups)

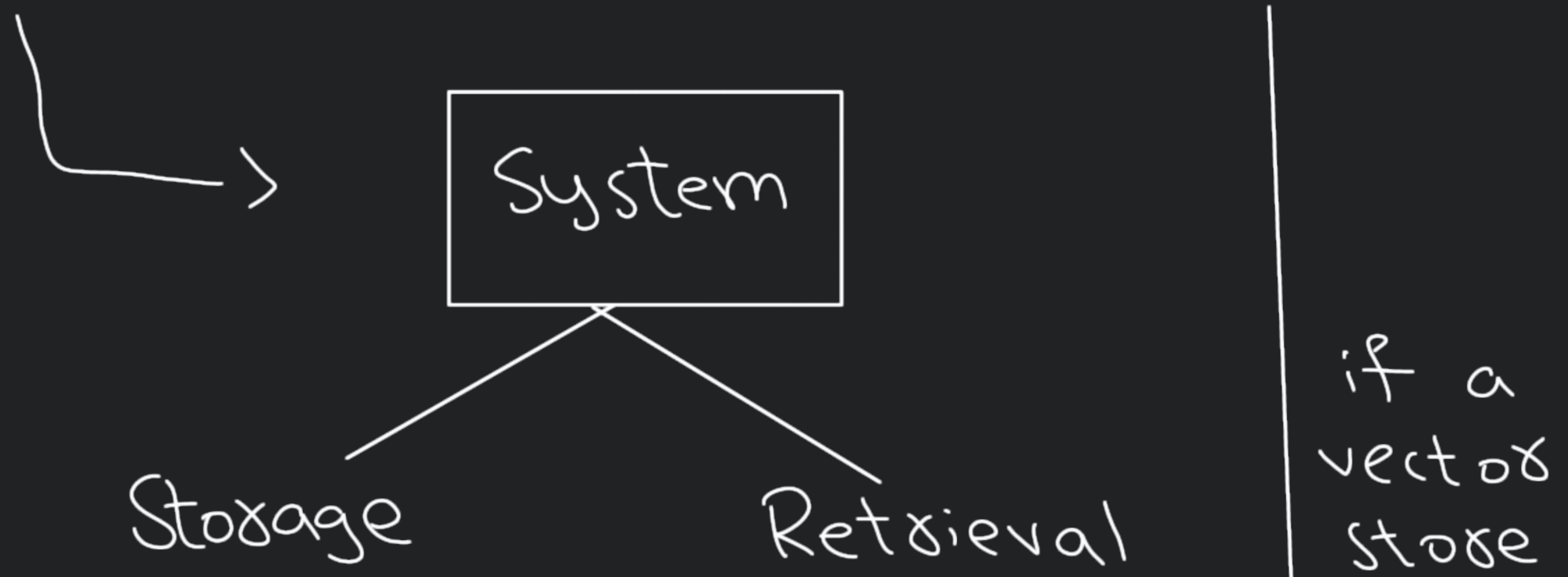
4) CRUD Operations - Manage the lifecycle of data - adding new vectors, reading them, updating existing entries, removing outdated vectors

## Use Cases:

- 1) Semantic Search
- 2) RAG
- 3) Recommender System
- 4) Image/Multimedia Search



# Vector Store vs Vector Database



→ Distributed

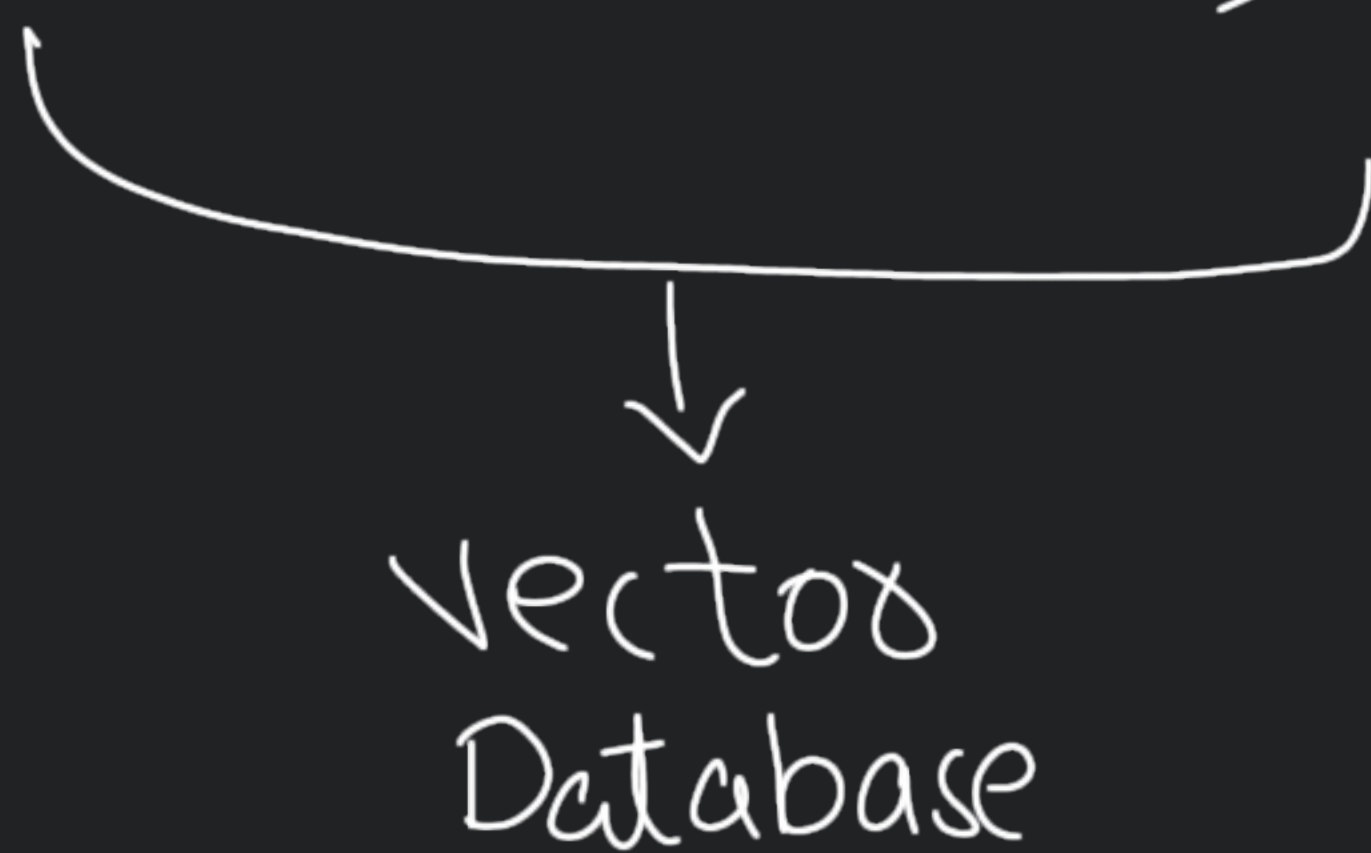
→ Backup and Restore

→ ACID Transactions

→ Concurrency

→ Authentication

Vector Store +

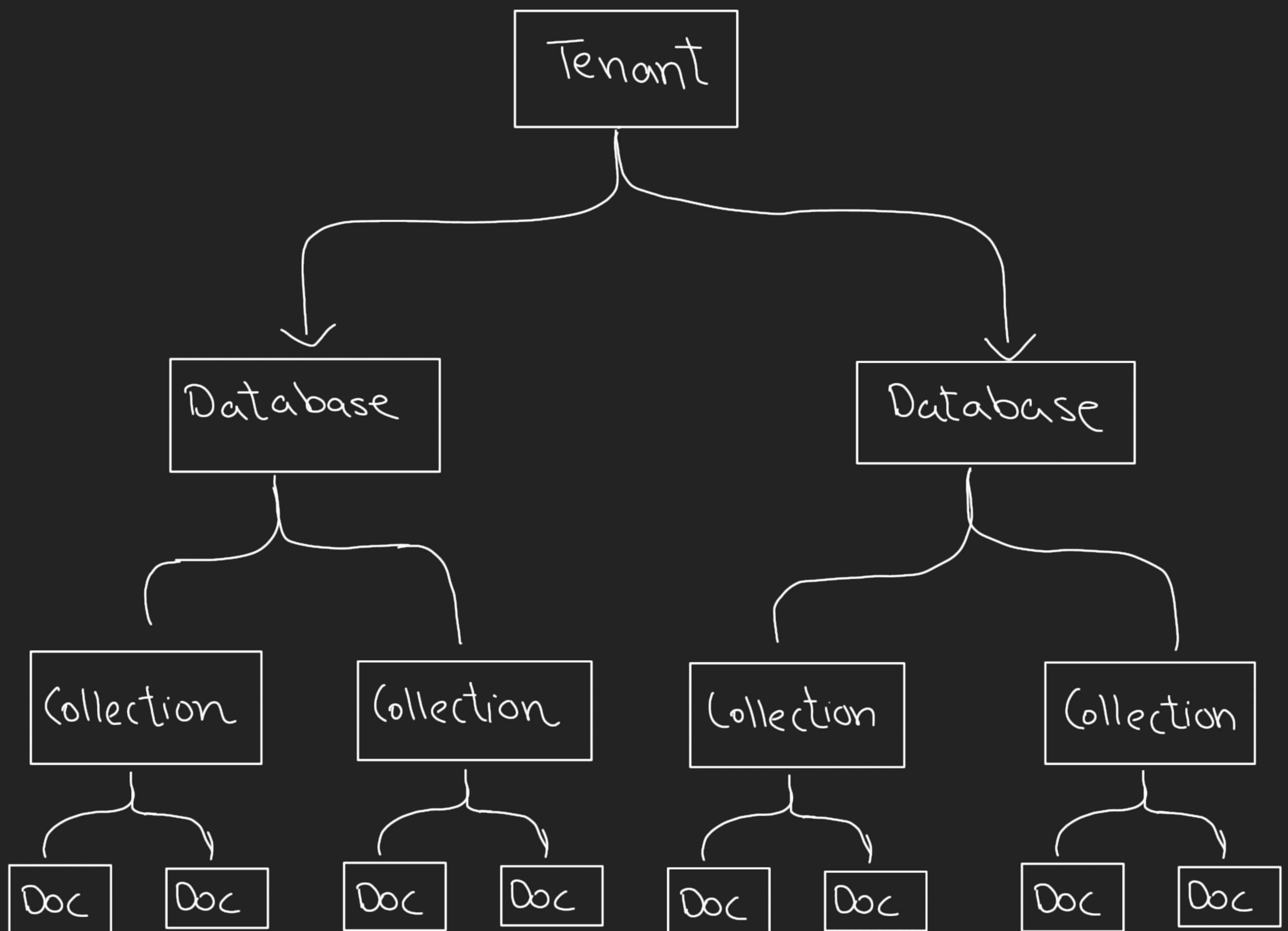




# Chroma Vector Store

Lightweight, open-source vector database that is especially friendly for local development and small-to medium scale production needs

## Chroma Tenancy and DB Hierarchy

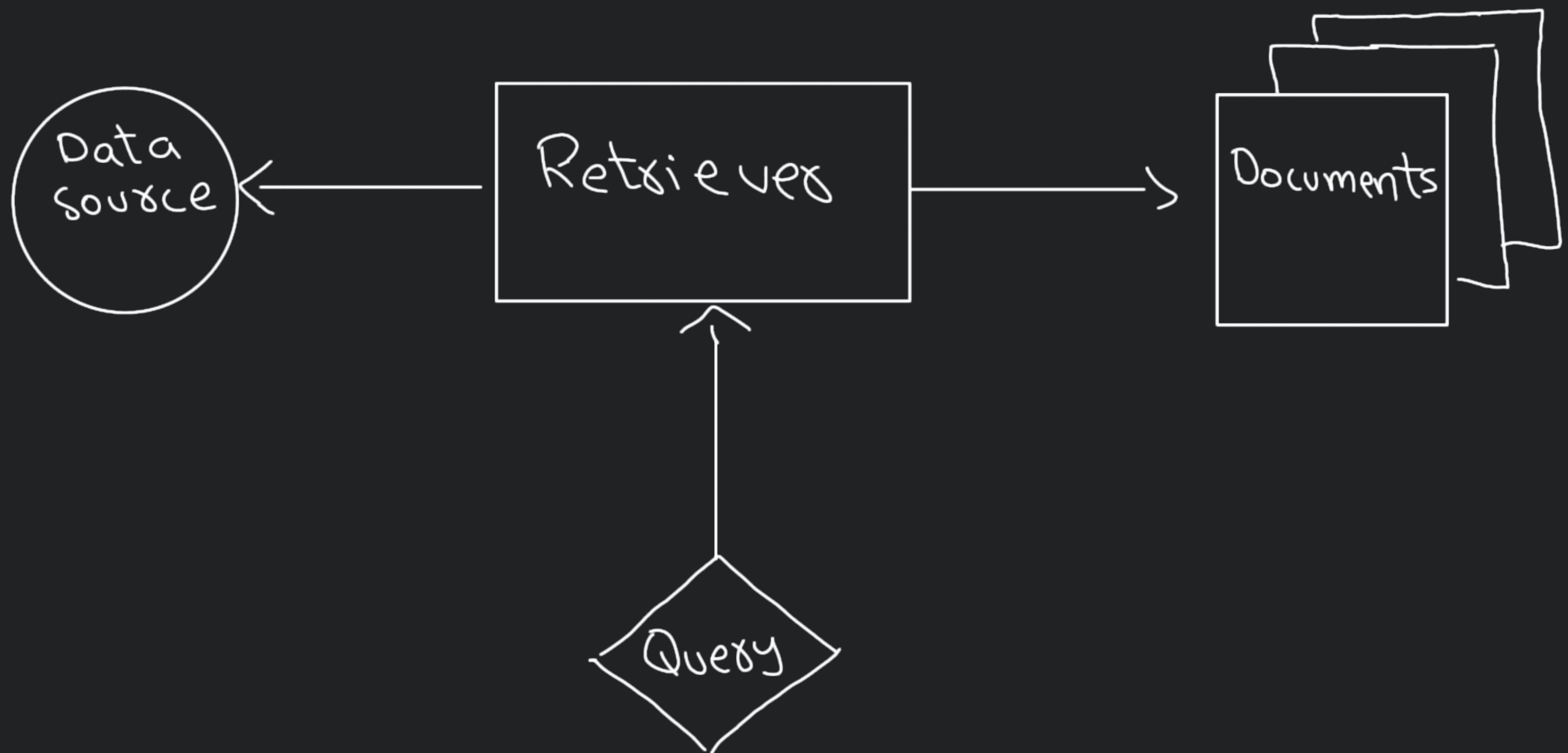




embedding metadata

## Retrievers

fetches relevant documents from a data source in response to a user's query



query  $\rightarrow f(x) \rightarrow$  multiple doc objects



# Types:

Categories:

- 1) Based on Data Sources
- 2) Search strategy

## 1) Based on Data Sources

Wikipedia Retriever: queries from wikipedia API to fetch relevant content

Vector Store Retriever: search and fetch documents from a vector store based on semantic similarity using vector embeddings

→ Similarity - Search in vector db has only 1 search strategy while retriever has different



and more advance search strategies

## 2) Search Strategy

### Maximal Marginal Relevance(MMR):

How can we pick results that are not only relevant to the query but also different from each other.

→ Informational retrieval algorithm designed to reduce redundancy in results while maintaining high relevance to the query

### Multi-Query Retriever:

Sometimes a single query might not capture all the ways information is phrased in your documents

- Takes original query
- Uses LLM to generate semantically different versions of that query



- Performs retrieval for each sub-query
- Combines and deduplicates the results

## Contextual Compression Retriever:

Advance retriever that improves retrieval quality by compressing documents after retrieval - keeping only relevant content based on user's query

